Contents

Chapter 1

Introduction

This document reports a simulation study that measures how much a bias-tailored code helps under biased noise — noise in which one kind of error occurs far more often than the others. It compares two closely related codes under identical biased noise:

- the **rotated CSS surface code**, the standard, bias-agnostic choice; and
- the **XZZX code**, a variant designed to exploit biased noise.

Both are studied across a range of bias strengths, from unbiased noise through to strongly biased noise, so that the effect of increasing bias can be read off directly.

The comparison uses two criteria, each reported as a function of the noise bias:

1. **Error-correction threshold** — the physical error rate below which making the code larger makes the logical qubit *more* reliable. A higher threshold means the code tolerates noisier hardware.
2. **Physical-qubit cost** — the number of physical qubits needed per logical qubit to reach a fixed target reliability. A lower cost means the same protection is achieved with fewer qubits.

The rest of the document is organised as follows. Chapter 2 describes how the two codes, the biased noise, and the experiments are constructed and simulated, relating each part of the method to its implementation. Chapter 3 explains how the error-correction threshold is extracted from the simulation data, and Chapter 4 how the physical-qubit cost is derived from it. Chapter 5 presents the measured thresholds and qubit costs for both codes across the full range of bias. Chapter 6 sets the findings against prior work and states the limitations of the study.

Chapter 2

Simulation Framework and Implementation

This chapter documents the simulation software used and, in particular, the bridge between the theory and the code that realises it. The threshold-estimation machinery is treated separately in chapter 3, and the physical-qubit cost derived from it in chapter 4.

2.1 Overview and toolchain

The simulator is built on the Stim stabiliser-circuit engine [10] for circuit construction, detector-error-model (DEM) extraction, and Monte-Carlo sampling, orchestrated across configurations by Sinter. Decoding uses PyMatching [13, 14] for minimum-weight perfect matching (MWPM), with two optional alternatives (section 2.5). Curve fitting uses `scipy.optimize.curve_fit`. The full dependency set is

- `stim 1.16`
- `sinter 1.16`
- `pymatching 2.4`
- `beliefmatching 0.2`
- `numpy`
- `scipy`
- `joblib`
- `pandas`
- `pandera`
- `ldpc`
- `matplotlib`.

Every experiment is a *memory experiment*: a logical state is prepared, held through r rounds of syndrome extraction, and read out, and the decoder is scored on whether it recovers the correct logical value. All experiments hold and read out the logical X observable (an *X-memory*), because the regime of interest is Z-biased noise, where the dominant errors are exactly those that the X-memory is sensitive to and where the XZZX code is expected to help. The physical error rate is a single scalar p; the noise bias is a second scalar η (section 2.3).

The control flow is:

`main.py`

→ `simulation.sweep`

→ `code_builder` + `circuit_builder`

→ `sinter.collect`

→ per-point logical error rates

→ `threshold.estimate_threshold` + `qubit_cost.qubit_cost_table`

→ figures.

The sweep grid — code family, distances, biases, and per-bias windows of p — is declared in `config.py`.

2.2 The codes: rotated surface and XZZX

Both codes are produced by `code_builder.py` and share the `QecCode` dataclass, which records the data- and ancilla-qubit layout on an integer lattice, the stabiliser support (a map from each ancilla to {data qubit → Pauli type}), the two logical operators, and a set `hset` of Hadamard-deformed data qubits. Generated lattice layout is later used by the code builders to build a Stim circuit.

Rotated surface code. For distance d, `build_rotated_surface_code` places the d^2 data qubits at odd–odd coordinates $(2c+1,\, 2r+1)$ with $r, c \in \{0, \ldots, d-1\}$ and the plaquette ancillas at even–even coordinates $(2c,\, 2r)$. Each plaquette acts on its (up to four) diagonal data neighbours. The stabiliser type is a checkerboard function of the plaquette position,

$$S(r,c) = \begin{cases} Z, & (r+c) \text{ even}, \\ X, & (r+c) \text{ odd}, \end{cases} \tag{2.1}$$

and weight-2 boundary checks are retained only where the standard rotated-code convention requires them (Z on the top/bottom edges, X on the left/right edges), with wrong-type corner checks discarded. The logical operators are a horizontal X string along the bottom row and a vertical Z string along the left column,

$$\overline{X} = \prod_{c=0}^{d-1} X_{(2c+1,\,1)}, \qquad \overline{Z} = \prod_{r=0}^{d-1} Z_{(1,\,2r+1)}. \tag{2.2}$$

XZZX code as a Hadamard deformation. The XZZX code is *not* constructed independently: `build_xzzx_code` takes the rotated surface code and conjugates a checkerboard sublattice of data qubits by a Hadamard. Writing $H : X \leftrightarrow Z,\ Y \to Y$, the deformed set is

$$\texttt{hset} = \{\, (2c+1,\, 2r+1) : (r+c) \text{ odd} \,\}, \tag{2.3}$$

and every stabiliser leg and logical leg on a qubit in `hset` has its Pauli type swapped $X \leftrightarrow Z$. A weight-4 plaquette that was pure X or pure Z thereby becomes a mixed $XZZX$ check. Because the two codes share the same data/ancilla layout, the same logical string positions, and the same syndrome-extraction schedule (section 2.4.4), the comparison isolates the effect of the deformation alone. For the CSS code `hset` is empty and all deformation logic reduces to the identity. The membership in `hset` also selects each data qubit's *effective preparation/measurement basis*, which is how the deformation is realised in the circuit (section 2.4).

2.3 Noise model and bias parameterisation

2.3.1 Single-qubit biased Pauli channel

Noise bias is defined, following the biased-noise literature [19, p. 2] [3, p. 4], as the ratio of the dephasing rate to the combined bit-flip rate,

$$\eta = \frac{p_Z}{p_X + p_Y}, \qquad p_X = p_Y, \qquad p = p_X + p_Y + p_Z, \tag{2.4}$$

so that the single-qubit channel used at every idle location is

$$p_X = p_Y = \frac{p}{2(1+\eta)}, \qquad p_Z = \frac{p\,\eta}{1+\eta}. \tag{2.5}$$

Here $\eta = 1/2$ is standard depolarising noise and $\eta \to \infty$ is pure dephasing, for which the channel degenerates to $(p_X, p_Y, p_Z) = (0, 0, p)$. This is `biased_pauli_rates(p, eta)` in `circuit_builder/shared.py`, applied as Stim's `PAULI_CHANNEL_1`.

2.3.2 Correlated biased two-qubit channel

Two-qubit gate faults use a correlated channel that generalises eq. (2.4). The 15 non-identity two-qubit Paulis are partitioned into a high-rate Z-subgroup $H = \{IZ, ZI, ZZ\}$ and the 12 remaining low-rate errors L, with bias $\eta = P(H)/P(L)$ and uniform probability within each set,

$$p_H = \frac{p\,\eta}{3(1+\eta)}, \qquad p_L = \frac{p}{12(1+\eta)}, \tag{2.6}$$

summing to p; as $\eta \to \infty$ the channel concentrates uniformly on H (each at $p/3$). This is `biased_two_qubit_rates(p, eta)`, emitted as `PAULI_CHANNEL_2` in the order Stim expects. A subtle but important consequence of the 3-vs-12 split is that the *two-qubit depolarising point is* $\eta = 1/4$, not $1/2$: all 15 components are equal when $p_H = p_L$, i.e. $\eta/3 = 1/12$. This asymmetry between the single- and two-qubit depolarising points is revisited when the low-bias thresholds are compared to the literature (chapter 6).

Justification and prior work. This parameterisation follows the circuit-level biased two-qubit channel of Darmawan *et al.* [4], who likewise keep the Z-type errors $\{ZI, IZ, ZZ\}$ at a high rate and damp the other twelve Paulis by the bias, of which eq. (2.6) is the symmetrised form. It presumes *bias-preserving* hardware — stabilised Kerr-cat qubits, whose bit-flips are exponentially suppressed — so that the two-qubit gate does not convert the dominant Z noise into X/Y errors; such a bias-preserving `CNOT` was demonstrated on cat qubits by Puri *et al.* [16], following Aliferis and Preskill [1]. On ordinary two-level qubits the boundary is sharper: the hybrid biased–depolarising model of Etxezarreta Martinez *et al.* [8] keeps the diagonal `CZ` biased (it commutes with Z, again the form of eq. (2.6)) but makes the `CNOT` near-depolarising, a no-go theorem capping its residual bias at $\eta_{\mathrm{CNOT}} \approx 5$. Applying the biased channel to the `CX` checks as well is thus the extra reach of cat hardware, and the validity of eq. (2.6) is conditional on this assumption (section 6.2).

2.4 Circuit model

Three circuit builders exist, all sharing a common `BaseCircuitBuilder`: a code-capacity model, a phenomenological model, and the circuit-level model. They form a hierarchy of realism: the code-capacity model is the most idealised — a single noise channel on the data qubits with otherwise perfect operations — whereas the circuit-level model is the most faithful, injecting noise at every individual operation (reset, two-qubit gate, idle, and measurement), with the phenomenological model in between. **Only the circuit-level model is exercised by the Monte-Carlo sweep.** The code-capacity builder is used solely to render circuit diagrams, and the phenomenolog-

Table 2.1: The three noise models provided by the circuit builders. Only the circuit-level model feeds the threshold sweep of this thesis.

Model	Data noise	Other noise	Rounds
Code capacity	one biased channel on data	none (perfect)	1
Phenomenological	bulk channel each round	measurement flips	d
Circuit-level	from operations only	reset + 2q gate + idle + measure	d

ical builder is present for reference but never run by default. Table 2.1 summarises the three.

2.4.1 Data-qubit initialisation and code-space projection

Every `build` method opens by initialising the data qubits in the *memory basis*, the basis of the logical observable held through the experiment. For the X-memory used throughout (section 2.1) this is the X basis, so `prep_data` prepares each off-`hset` data qubit in $|+\rangle$ with `RX`. Under the XZZX deformation the frame is rotated on the Hadamard sublattice, so the `hset` qubits are instead prepared in $|0\rangle$ with `R` (the Z basis); `data_basis_partition` computes the split once from `prep_basis_of` (section 2.2), the same per-qubit basis assignment that realises the deformation. For the CSS code `hset` is empty and all data qubits start in $|+\rangle$.

Preparing in the memory basis makes the held logical operator deterministic: the initial product state is a $+1$ eigenstate of the logical $\overline{X}$ string of eq. (2.2), so the encoded logical value is definite from the outset. For the same reason it is a $+1$ eigenstate of every *same-type* check — the X-type stabilisers and their XZZX-deformed images, which are exactly the *eligible* checks of section 2.4.5 — since each such check acts on its legs only with the operator those qubits were prepared to diagonalise.

The product state is *not*, however, an eigenstate of the complementary (Z-type) checks, whose outcomes on it are random. The first round of syndrome extraction measures all stabilisers, and this measurement *projects* the state into a simultaneous eigenstate of the full stabiliser group — that is, into the code space — fixing those random outcomes into a reference syndrome. Because every later round is compared against the previous one (`consecutive_round_detectors`) or reconstructed at the boundaries (section 2.4.5), the random initial values cancel and only *changes* caused by faults are detected. This is why round 0 serves as the reference that projects into the code space: in the code-capacity and phenomenological builders it is noiseless, whereas in the circuit-level builder round 0 is itself noisy and carries the preparation reset error (table 2.2), so its initial time boundary must instead be closed explicitly through the deterministic eligible checks (sections 2.4.5 and 2.4.7).

2.4.2 Syndrome extraction gadget

A single uniform gadget measures any $X/Z/Y$ stabiliser leg mix: the ancilla is prepared in $|+\rangle$, a controlled-Pauli (`CX`/`CY`/`CZ`, ancilla as control) is applied for each leg, and the ancilla is measured in the X basis, so that phase kickback deposits the stabiliser eigenvalue onto the ancilla. The same code path therefore serves both CSS and XZZX checks; the mixed XZZX legs simply select a different controlled gate per leg via `CGATE = {X:CX, Y:CY, Z:CZ}`. Detectors compare each ancilla between consecutive rounds, so a silent detector means the syndrome did not change. Figure 2.1 shows this gadget for the surface-code X and Z stabilisers, and fig. 2.2 for the mixed XZZX check.

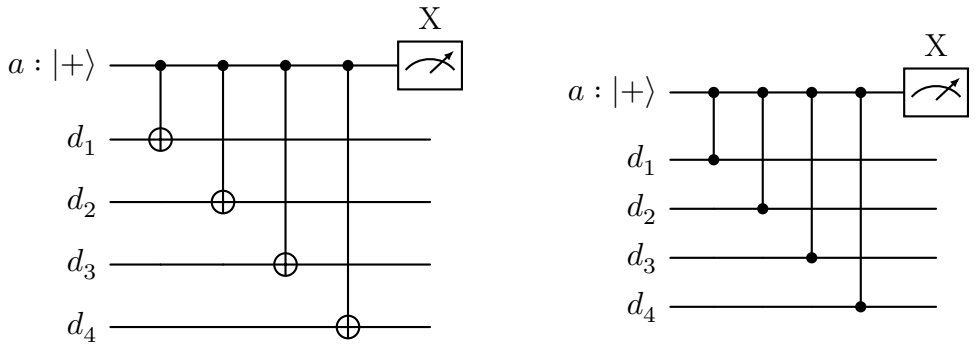

Figure 2.1: Syndrome-extraction gadget for a surface-code ancilla. The ancilla is prepared in $|+\rangle$, one controlled Pauli is applied per leg (ancilla as control), and the ancilla is measured in the X basis. Left: an X-stabiliser, four `CX` gates. Right: a Z-stabiliser, four `CZ` gates.

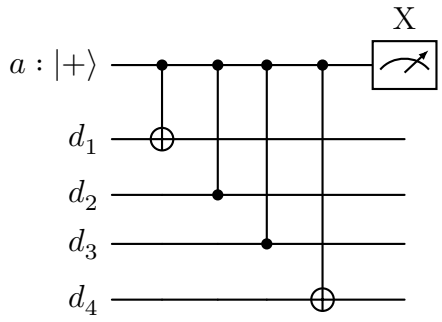

Figure 2.2: Syndrome-extraction gadget for an XZZX ancilla. The same prepare-$|+\rangle$, controlled-Pauli, measure-X gadget now mixes gate types: the schedule order yields `CX`, `CZ`, `CZ`, `CX` (i.e. $X\,Z\,Z\,X$), with the X legs on one diagonal and the Z legs on the other.

The Hadamard deformation is realised through this per-qubit preparation and measurement basis (section 2.4.1): each data qubit is likewise read out in the basis its frame requires, `MX` off `hset` and `M` on `hset`.

Table 2.2: Fault locations of one circuit-level round, with the Stim instruction and the physical origin. Rates are (p_X, p_Y, p_Z) from eq. (2.5) for the single-qubit channels and eq. (2.6) for the two-qubit channel.

Location	Instruction	Origin
Ancilla reset	`Z_ERROR(p)` after `RX`	imperfect $\|+\rangle$ preparation (Z flips it)
Two-qubit gate	`PAULI_CHANNEL_2` after `CX`/`CZ`	biased, bias-preserving entangling fault
Idle (reset window)	`PAULI_CHANNEL_1` on all data	data waits while ancillas reset
Idle (per CNOT step)	`PAULI_CHANNEL_1` on ungated qubits	boundary qubits rest during a step
Idle (measure window)	`PAULI_CHANNEL_1` on all data	data waits while ancillas are measured
Measurement	`MX(p_meas)`	readout flip ($p_{\text{meas}} = p$ by default)

2.4.3 Circuit-level (biased SD6) fault locations

The circuit-level model is a *biased SD6* model: the single-parameter SD6 convention in which every operation is noisy, with each channel made Z-biased by η. Crucially it assumes *bias-preserving two-qubit gates*, as realisable on e.g. Kerr-cat qubits [4], so both the `CZ` (Z-type checks) and `CX` (X-type checks) gates receive the *same* biased correlated channel of eq. (2.6). Table 2.2 lists the six fault locations of one round, as implemented in the overridden `syndrome_meas` of `circuit_level.py`.

There is deliberately *no* bulk per-round data channel: all data decoherence comes from the reset, gate, and idle faults, so adding a phenomenological bulk term would double-count it. The number of rounds defaults to the code distance, $r = d$, the standard convention for circuit-level surface-code memories [6, p. 8] [9, p. 45], giving the time direction the same distance-d redundancy as the space directions.

Both time boundaries are noisy: round 0 is itself a noisy round closed by `initial_boundary_detectors`, and the final data readout carries a readout error closed by `final_boundary_detectors` (section 2.4.5). Leaving round 0 noisy (rather than using a perfect reference round) is essential — a reset fault placed before a perfect round 0 would be absorbed into the reference and collapse the effective distance to ~ 1.

2.4.4 CNOT schedule and distance preservation

The per-leg controlled gates for syndrome extraction are scheduled into *four parallel time steps* by `build_cnot_schedule`, which assigns each leg to a step from its lattice offset $(\Delta x, \Delta y)$ relative to the ancilla, using one of two offset→step maps selected by plaquette parity. The *order* of these steps matters because of *hook errors*: a single

fault on an ancilla partway through its sequence of controlled gates propagates onto every data qubit gated *after* it, so one fault can produce a correlated weight-2 data error rather than a harmless weight-1 one [6]. If the gate order is chosen carelessly these weight-2 errors can line up along the logical operator, so that only $\sim d/2$ of them suffice to span the code, halving the effective distance. A fixed, carefully chosen schedule is therefore needed to orient hook errors transverse to the logical direction. Two properties are enforced:

1. **No double-booking:** within a step no data qubit is targeted by two ancillas, since the four neighbours of a data qubit sit at four distinct offsets.

2. **Distance preservation:** the fixed offset-to-step assignment keeps weight-2 *hook errors* off the logical-X direction, so a single mid-round ancilla fault cannot align with the logical operator and halve the effective distance.

This is verified operationally at run time: `validate_distance_preservation` builds each circuit at $p = 0.1$ and asserts that `circuit.shortest_graphlike_error()` has length exactly d for every $(\eta, \text{code}, d, \text{basis})$.

2.4.5 Time-boundary detectors: initial and final

Consecutive-round detectors compare a syndrome against the previous round, which leaves the two ends of the round sequence open: the first round has no predecessor, and the final data state is read out destructively rather than via ancillas. The time boundaries are closed by detectors reconstructed from operations that are deterministic under the chosen preparation.

Both boundary methods first compute the *eligible* set — the stabilisers every leg of which matches its data qubit's preparation basis (`prep_basis_of`, section 2.2), i.e. the checks that are deterministically +1 under the initial product state and can be reconstructed from the final single-qubit data readout. For an X-memory these are the X-type checks (and their XZZX-deformed counterparts).

This is the standard way of closing the two open time boundaries of a memory experiment: it is the detector formalism of Stim, in which a detector is a set of measurements whose parity is deterministic in the absence of noise [10], applied to the surface-code memory protocol in which the data product state fixes the first-round stabilisers and the final transversal data readout reconstructs the last-round ones [11]; conceptually it realises the explicit time boundaries of the (2+1)-dimensional space-time picture of Dennis *et al.* [6].

Initial boundary (bottom). `initial_boundary_detectors` emits, for each eligible ancilla, a *single-measurement* detector on that ancilla's round-0 syndrome. Because

the eligible checks are +1 deterministically on the prepared state, any round-0 deviation — a reset fault or a round-0 readout flip — makes the lone measurement fire. This boundary is used *only* by the circuit-level model, whose round 0 is noisy; it is what makes the reset error (table 2.2) detectable.

Final boundary (top). `final_boundary_detectors` emits, for each eligible ancilla, a detector on the *reconstructed* check — the product of the final data-readout results over that check's legs — together with the ancilla's last-round syndrome. Agreement means no error struck between the last round and the destructive readout; a mismatch flags one, so errors near the end of the experiment (including data-readout flips at rate p_{meas}) are caught. An initial detector therefore references one measurement, whereas a final detector references $w + 1$ (a weight-w check plus its last-round syndrome); the initial boundary is the mirror image of the final one at the bottom of the circuit.

2.4.6 The logical observable

The last step of every `build` method, `define_observable`, records *which* logical value the experiment must preserve, so that the decoder can be scored against it. It selects the logical operator matching the memory basis — `logical_x` for the X-memory (or `logical_z` for a Z-memory) — and emits a single Stim `OBSERVABLE_INCLUDE` (index 0) over the final data-readout records of that operator's support:

$$o_0 \;=\; \bigoplus_{q \,\in\, \operatorname{supp} \overline{X}} m_q,$$

where o_0 is the recorded observable and m_q the final single-qubit readout of data qubit q, obtained from `data_readout` and addressed through `data_record` as a relative measurement record.

Because $\overline{X}$ is a *product of single-qubit Paulis* along the bottom row (eq. (2.2)) and each data qubit is read out transversally in exactly the basis that operator requires — the X basis (`MX`) off `hset` and the Z basis (`M`) on `hset`, so that the deformed $\overline{X}$ of the XZZX code is reconstructed leg by leg — the eigenvalue of $\overline{X}$ is precisely the parity of those readouts. That parity is deterministic in the absence of errors, since the data were prepared as a +1 eigenstate of $\overline{X}$ (section 2.4.1); `OBSERVABLE_INCLUDE` declares exactly this determinism to Stim, which is what lets a decoder be graded on it.

A logical error is then a *flip* of this observable that the decoder fails to predict: for each shot the decoder outputs a predicted parity from the syndrome, Sinter compares it against the recorded o_0, and a disagreement counts as one logical failure — the events tallied by the logical error rate p_L of section 2.5. A single observable is defined, matching the one logical qubit of the memory experiment.

2.4.7 Circuit assembly: the `build` methods

Each builder exposes a `build` method that assembles the full Stim circuit for one memory experiment. All three share the opening (declare qubit coordinates, then prepare the data in memory basis via `prep_data`) and the closing (a final data readout, the logical observable via `define_observable`), and differ in the reference round, the per-round noise, and the time-boundary handling.

Code capacity:

`prep_data`

→ a perfect `syndrome_meas()`
(round 0, the reference that projects into the code space)

→ a single biased `PAULI_CHANNEL_1` on the data

→ a second perfect `syndrome_meas()` (round 1)

→ `consecutive_round_detectors` comparing round 1 against round 0

→ a perfect `data_readout` and the observable.

There are exactly *two* syndrome rounds and no time-boundary detectors.

Phenomenological:

`prep_data`

→ a perfect reference round 0

→ `repeat_rounds(`d`)`, each round applying a bulk `data_round_noise` channel, then `syndrome_meas(flip` $= p_{\texttt{meas}}$`)`, then `consecutive_round_detectors`

→ a perfect `data_readout`

→ `final_boundary_detectors`

→ the observable.

The rounds are emitted as one Stim `REPEAT` block by `repeat_rounds`, which fixes up the measurement-record bookkeeping so that the relative offsets and the `SHIFT_COORDS` time coordinate are identical in every iteration.

Table 2.3: Circuit assembly across the three models. A is the number of ancillas (stabilisers) and d the code distance; the multi-round models default to d rounds.

Model	Round 0	Noisy rounds	Boundary detectors	Ancilla meas.
Code capacity	perfect	1	none	$2A$
Phenomenological	perfect	d	final	$(1+d)A$
Circuit-level	noisy	d	initial + final	dA

Circuit-level:

`prep_data` followed by an explicit reset error (`Z_ERROR(`p`)` on the $|+\rangle$-prepared qubits, `X_ERROR(`p`)` on the $|0\rangle$-prepared qubits)

$\rightarrow$ a *noisy* round 0 via the overridden `syndrome_meas` (table 2.2)

$\rightarrow$ `initial_boundary_detectors`

$\rightarrow$ `repeat_rounds(`$d-1$`)`, each a noisy `syndrome_meas` followed by `consecutive_round_detectors`

$\rightarrow$ `data_readout(flip` $= p_{\texttt{meas}}$`)`

$\rightarrow$ `final_boundary_detectors`

$\rightarrow$ the observable.

Because round 0 is itself one of the noisy rounds, only $d-1$ further rounds are repeated, for d noisy syndrome rounds in total and *no* perfect reference.

Measurement rounds: code capacity versus the multi-round models. The number of syndrome-measurement rounds is the sharpest structural difference between the models (table 2.3). With perfect measurements (code capacity) a single post-noise readout is trustworthy, so the circuit needs only the reference round plus one measured round — *two rounds regardless of* d. Once measurements can fail (phenomenological and circuit-level), a lone syndrome cannot distinguish a data error from a measurement error, so the experiment is repeated for $\mathcal{O}(d)$ rounds to give the time direction the same distance-d redundancy as the two spatial directions.

2.5 Decoding and the logical error rate

For each configuration the Stim circuit is converted to a DEM with `decompose_errors=True, approximate_disjoint_errors=True`; the second flag is required because the correlated `PAULI_CHANNEL_2` faults must be decomposed into graphlike edges for a matching decoder. Three decoders are available (table 2.4); MWPM is the default.

Table 2.4: Available decoders. MWPM (default) is used for all reported thresholds.

Flag	Decoder	Backend
`mwpm`	minimum-weight perfect matching	PyMatching [7, 13]
`bp`	belief propagation + matching	beliefmatching [15]
`uf`	Union-Find (peeling)	`ldpc` [5, 17]

The logical error rate is $p_L = k/N$ for k decoding failures in N shots. Sampling is adaptive: shots are drawn until either `target_errors` failures are seen or `max_shots` is reached. The reported per-point uncertainty σ is the half-width of the Wilson score interval at $z = 1$ ($\approx 1\sigma$), which is well-behaved as $p_L \to 0$ and is reused verbatim as the fit weight in chapter 3 and as the plotted error bar.

MWPM is the standard fast decoder but is *not optimal* here: forcing the correlated two-qubit noise into a graphlike DEM discards the residual edge correlations, and under strong Z-bias the dominant errors are highly correlated along the deformed lattice. Every reported threshold is therefore a *lower bound* on what a correlation-aware decoder (correlated matching, or BP+OSD) would achieve; this caveat is quantified in chapter 6.

2.6 Theory-to-implementation map

Table 2.5 collects the correspondence between the theoretical objects of the earlier chapters and their concrete realisation in the source, as an index for verification.

Table 2.5: Correspondence between theory and implementation. Line-level details are given in the sections referenced in the last column.

Theoretical object	Implementation	Reference
Rotated surface code, distance d	`build_rotated_surface_code`	section 2.2
X/Z checkerboard, eq. (2.1)	`(r+c)%2` type assignment	section 2.2
XZZX Hadamard deformation, eq. (2.3)	`build_xzzx_code`, `deform`, `hset`	section 2.2
Biased single-qubit channel, eq. (2.5)	`biased_pauli_rates`	section 2.3
Biased two-qubit channel, eq. (2.6)	`biased_two_qubit_rates`	section 2.3.2
Circuit-level fault locations	`CircuitLevelCircuitBuilder.syndrome_meas`	section 2.4
Distance-preserving schedule	`build_cnot_schedule` + `validate_distance_preservation`	section 2.4.4
Syndrome $\rightarrow$ correction (MWPM)	`resolve_decoder`, PyMatching	section 2.5
Wilson 1σ error bar	`wilson_interval`	section 2.5
FSS threshold fit	`estimate_threshold`	chapter 3
Physical-qubit cost	`qubit_cost_table`, `required_distance`	chapter 4

Chapter 3

Threshold Estimation via Finite-Size Scaling

The error-correction threshold p_{th} is the physical error rate below which increasing the code distance suppresses the logical error rate. This chapter documents how p_{th} is extracted from the Monte-Carlo data of chapter 2: the finite-size-scaling (FSS) hypothesis behind it, the polynomial ansatz implemented in `threshold.py`, the windowed fitting procedure, the uncertainty estimate, and the limitations of the method.

3.1 The threshold as a phase transition

Fix the noise model and bias η and sweep the physical error rate p and the code distance d. Each shot succeeds or fails, giving a logical error rate $p_L(p, d)$. A code family has a sharp threshold if

$$\lim_{d\to\infty} p_L(p,d) = \begin{cases} 0, & p < p_{\text{th}}, \\ \text{const} > 0, & p > p_{\text{th}}. \end{cases} \tag{3.1}$$

This crossover is a continuous phase transition: p_{th} plays the role of the critical point, $p - p_{\text{th}}$ that of the reduced temperature, and d that of the system size [6, 21]. At $p = p_{\text{th}}$ the logical error rate is asymptotically independent of d, so the raw p_L-versus-p curves for different distances cross at a single point — the first visual estimate of the threshold, and the crossing whose movement with η is the object of the CSS-versus-XZZX comparison.

3.2 The finite-size-scaling ansatz

Near a continuous transition the correlation length diverges as $\xi(p) \sim |p - p_{\text{th}}|^{-\nu}$ with critical exponent ν. Universality then implies that a finite system of size d depends on

p and d only through the ratio $d/\xi \sim (p - p_{\text{th}})\, d^{1/\nu}$, giving the FSS ansatz

$$p_L(p, d) = F(x), \qquad x = (p - p_{\text{th}})\, d^{1/\nu}, \tag{3.2}$$

with a single universal scaling function F shared by all distances. Two consequences drive the method: with the correct (p_{th}, ν) the points from every distance *collapse* onto one curve $F(x)$, and at $x = 0$ they *cross*. For the 2D surface code under independent noise, $\nu \approx 1.5$ (random-bond-Ising / percolation universality) [2, 21], so a fitted ν far from $\sim$ 1–1.5 is a warning sign rather than a physical result.

3.3 Polynomial ansatz

F has no closed form, so following standard practice [21, p. 11-12] it is Taylor-expanded near threshold — here to second order:

$$p_L(p, d) \approx a + b\, x + c\, x^2, \qquad x = (p - p_{\text{th}})\, d^{1/\nu}. \tag{3.3}$$

This is `fss(X, p_th, nu, a, b, c)` in `threshold.py`, with 5 free parameters $(p_{\text{th}}, \nu, a, b, c)$.

The Taylor truncation is valid only for small $|x|$. Far below threshold p_L decays roughly exponentially in d and far above it saturates, neither of which is quadratic in x; including such points would bend the parabola and pull the apparent p_{th}. The fit is therefore *windowed* in x and restricted to the largest distances (section 3.5).

3.4 The statistical model of each point

For a configuration (p, d) the simulator draws N shots and counts k logical failures, so $k \sim \text{Binomial}(N, p_L)$ and $\hat{p}_L = k/N$. Sampling stops on a fixed error count (`target-errors`) or a shot cap (`max-shots`); stopping on errors keeps the relative statistical uncertainty $\approx 1/\sqrt{k}$ roughly constant across points spanning orders of magnitude in p_L. The uncertainty σ is the half-width of the Wilson score interval at z standard deviations,

$$\sigma = \frac{z}{1 + z^2/N} \sqrt{\frac{\hat{p}(1-\hat{p})}{N} + \frac{z^2}{4N^2}}, \qquad \hat{p} = \frac{k}{N}, \tag{3.4}$$

evaluated at $z = 1$. Unlike the naive Bernoulli error $\sqrt{\hat{p}(1-\hat{p})/N}$, this stays inside $[0, 1]$ and does not collapse to zero as $p_L \to 0$ — the deep-sub-threshold regime that dominates the sweep. Points with $k = 0$ carry no usable two-sided σ and are dropped before fitting.

3.5 Fitting procedure and windowing

`estimate_threshold` fits eq. (3.3) with `scipy.optimize.curve_fit`, minimising the weighted chi-square $\chi^2 = \sum_i \left[(\hat{p}_{L,i} - \text{model}_i)/\sigma_i\right]^2$.

1. **Restrict to the largest distances.** Only $d \in \{11, 13, 15, 17, 19\}$ enter the fit; the smaller distances, which carry the largest finite-size corrections, are excluded outright rather than modelled.

2. **Seed the fit.** A crossing seed $p_{\text{th}}^{(0)}$ is taken as the p at which the p_L values of the retained distances agree most closely — the p minimising the across-distance standard deviation of a p-by-d pivot table (`crossing_seed`) — and the critical exponent is seeded at $\nu^{(0)} = 1.1$.

3. **Fixed rescaled window.** With the seed, the fit is restricted to the fixed window $|x| \le x_{\max}$ with $x_{\max} = 0.006$, evaluated in the *rescaled* variable $x = (p - p_{\text{th}}^{(0)})\, d^{1/\nu^{(0)}}$ computed from the seed. Bounding $|x|$ (rather than p) controls the $O(x^3)$ truncation error uniformly across distances, whereas a window in p would retain exactly the large-d points that break the quadratic first.

4. **Single fit.** The retained points are fit once for $(p_{\text{th}}, \nu, a, b, c)$, seeded at $(p_{\text{th}}^{(0)}, \nu^{(0)}, \text{median}\, \hat{p}_L, 0, 0)$.

The fit passes `absolute_sigma=True` so that the returned covariance is a genuine statistical covariance, and bounds $p_{\text{th}} \in (0, 1)$ and $\nu \in (1, 2)$ keep the $d^{1/\nu}$ factor well behaved and ν in the physically plausible band around the random-bond-Ising value. The reduced chi-square χ^2_{red} is reported over $N_{\text{win}} - 5$ degrees of freedom, together with its survival-function p-value $P(\chi^2_{\text{dof}} \ge \chi^2_{\text{red}} \cdot \text{dof})$ — the probability that a correct model would produce a chi-square at least this large. A p-value near or above conventional significance means the quadratic ansatz is statistically consistent with the data, whereas a small p-value flags a poor fit; this p-value is displayed in the data-collapse figure of each fit (section 3.7).

3.6 Uncertainty estimate

The reported 1σ error bar on p_{th} is the *most conservative* of three quantities:

$$\delta p_{\text{th}} = \max\Big(\delta p_{\text{th}}^{\text{cov}},\ \ \delta p_{\text{th}}^{\text{cov}} \sqrt{\chi^2_{\text{red}}},\ \ \delta p_{\text{th}}^{\text{boot}} \Big). \tag{3.5}$$

- **Covariance:** $\delta p_{\text{th}}^{\text{cov}} = \sqrt{(\text{pcov})_{00}}$, exact only if the linearised model holds at the optimum.

- **Chi-square-inflated covariance:** $\delta p_{\mathrm{th}}^{\mathrm{cov}} \sqrt{\chi^2_{\mathrm{red}}}$ widens the covariance bar whenever the fit is worse than statistically perfect ($\chi^2_{\mathrm{red}} > 1$), absorbing mild model misspecification into the reported error.

- **Parametric bootstrap:** for $n_{\mathrm{boot}} = 5000$ replicas, each windowed point's failure count is resampled as $k_i^* \sim \mathrm{Binomial}(N_i, \hat{p}_{L,i})$ and the fit is repeated (seeded at the point estimate); $\delta p_{\mathrm{th}}^{\mathrm{boot}}$ is the standard deviation of the resampled p_{th}^*. The bootstrap RNG is seeded for reproducibility.

Taking the maximum makes the quoted uncertainty robust: it is never smaller than the linear-covariance estimate, and is widened by either a poor fit or a heavy-tailed resampled distribution. The `FitResult` carries $p_{\mathrm{th}} \pm \delta p_{\mathrm{th}}$, ν, the polynomial coefficients (a, b, c), the reduced chi-square χ^2_{red}, its p-value, the window half-width $x_{\max}$, and the list of fitted distances.

3.7 Reading the data-collapse figure

The primary validation plot (`collapse_<eta>.pdf`, produced by `draw_collapse`) rescales each point to $x = (p - p_{\mathrm{th}})\, d^{1/\nu}$ and plots it against the logical error rate p_L, keeping only the fitted distances and the points inside the window $|x| < x_{\max}$, with the fitted parabola $a + bx + cx^2$ overlaid. A clean collapse of all distances onto one curve, together with $\chi^2_{\mathrm{red}} \approx 1$, is the evidence for the quoted $p_{\mathrm{th}} \pm \delta$; distances peeling into separate branches, or a large χ^2_{red}, signal a window that is too wide, distances that are too small, or a genuine breakdown of the simple scaling form.

3.8 Assumptions and limitations

The following caveats are recorded explicitly, as they bound how the numbers of chapter 5 may be interpreted:

- **X-memory threshold only.** The quoted p_{th} is the X-memory threshold under Z-biased noise, not a full (X and Z) code threshold. This is nonetheless the relevant basis for the comparison, because the X-memory is the worst case under Z-biased noise: the logical-X observable fails through logical-Z-type errors, which are assembled from exactly the dominant physical Z (and Y) faults, so the X-memory carries a higher logical error rate — and hence a lower threshold — than the Z-memory, whose logical failures require the strongly suppressed physical X/Y errors. Since a logical qubit's memory lifetime is set by whichever basis fails first, the X-memory is the rate-limiting one, and reporting and comparing the codes on it is both conservative and sufficient for establishing the CSS-versus-XZZX ordering.

- **MWPM lower bound.** Thresholds are decoder-dependent and MWPM is sub-optimal for the correlated, biased noise here, so every value is a lower bound (section 2.5).

- **Small distances excluded, not corrected.** Small d carry large finite-size corrections. The active fit does not model them; instead it discards them, fitting only the largest distances $d \in \{11, \ldots, 19\}$. This trades statistical reach for a cleaner quadratic, and leaves the residual finite-size systematic of the retained distances as an unmodelled contribution; larger distances and denser sampling near the crossing remain the real remedies.

- **Fixed window and seed.** The window half-width $x_{\max} = 0.006$ and the seed exponent $\nu^{(0)} = 1.1$ are fixed rather than selected per fit. This keeps the procedure simple and reproducible, but the result is not insulated against a poorly matched window: a window too wide for a given (η, code) shows up as an inflated χ^2_{red} and a correspondingly widened error bar via eq. (3.5).

- **Finite shot budget versus the model floor.** No finite parametric form is exact, so as $\sigma_i \to 0$ (infinite shots) χ^2_{red} must eventually exceed 1; the reported values reflect the quadratic ansatz and fixed window at the *present* budget, not a claim of exactness.

- **Bias-agnostic collapse.** The same scaling form is used for every η. For strongly biased XZZX this degrades: χ^2_{red} stays above 1 with a correspondingly widened bootstrap bar. The fit still *locates* the threshold, but is not a high-precision determination of p_{th} or ν there. A bias-dependent scaling function is the natural next refinement.

- **Finite-size corrections at high bias ($d < \eta$).** This is not merely a limitation of the present ansatz but a known property of tailored codes. Xiao, Srivastava and Granath [22] show, using exact finite-size results, that for surface codes tailored to biased noise the corrections when the (Z-error) distance is below the bias, $d_Z < \eta$, are large enough that threshold extractions from numerical decoder data at moderate code distances are *unreliable*; in this regime the total logical failure rate tends to *overestimate* the threshold (while the bit-flip rate underestimates it), and a confident value requires either resolving the phase- and bit-flip thresholds separately or reaching distances comparable to the bias (they use $d \approx 100$). The present sweep sits squarely in this regime for the strongly biased XZZX cases: at $\eta \gtrsim 30$ every fitted distance ($d \leq 19$) satisfies $d < \eta$, and the experiment records a single total logical-X failure rate. The high-bias XZZX entries of table 5.3 should therefore be read as *indicative*, consistent-with-the-trend values rather than converged threshold determinations.

- **Per-model p-axis.** Thresholds from different noise models live on different p-axes and must not be compared at the same numeric p; only the CSS-versus-XZZX comparison within one model and one η is meaningful.

Chapter 4

Physical-Qubit Cost Estimation

A threshold answers whether a code family suppresses errors at a given physical error rate p, but not how expensive that suppression is. This chapter documents the second post-processing stage, `qubit_cost.py`, which converts the sub-threshold behaviour of each (η, code) group into a single operational number: the physical qubits per logical qubit required to reach a target logical error rate at a fixed operating point. Because both codes are square rotated surface codes with the same qubit count at a given distance, this cost is a direct, bias-resolved measure of the XZZX advantage that complements the thresholds of chapter 3.

4.1 Sub-threshold suppression law

Below threshold the logical error rate of a surface-code memory falls exponentially in the code distance [6]. Writing the per-cycle logical error rate as ε, the standard scaling form is $\varepsilon \propto (p/p_{\text{th}})^{\kappa d}$, so that

$$\ln \varepsilon = \ln A + \kappa\, d\, \ln \frac{p}{p_{\text{th}}}, \tag{4.1}$$

which is linear in d with a slope set by how far below threshold the operating point sits. This is the empirical sub-threshold law $P_L \approx A\,(p/p_{\text{th}})^{d_e}$ of Fowler *et al.* [9], whose fault-counting exponent is $d_e = \lfloor (d+1)/2 \rfloor$. The suppression exponent prefactor is fixed at $\kappa = 0.5$ (`KAPPA`), giving $\kappa d = d/2$; this differs from $(d+1)/2$ by the constant $1/2$, which is absorbed into the prefactor A and so does not affect the fitted slope. Fixing κ rather than fitting it is justified because $d/2$ is not a free parameter but the leading-order suppression exponent set by the code's minimum-weight logical operator — a distance-d surface code needs $\lceil d/2 \rceil$ simultaneous faults to produce an undetected logical error — so imposing it leaves only the prefactor $\ln A$ to be estimated from the limited sub-threshold data and avoids overfitting a second, weakly constrained parameter. With $p < p_{\text{th}}$ the log-ratio $\ln(p/p_{\text{th}})$ is negative, so $\ln \varepsilon$ decreases with d as

required.

This is exactly the exponential-suppression law of the Google Quantum AI surface-code experiments [11, 12]: eq. (4.1) with $\kappa = 0.5$ is written there [12, Eq. (1)] as $\varepsilon_d \propto (p/p_{\text{th}})^{(d+1)/2}$, with the per-cycle error suppressed by a factor $\Lambda \equiv \varepsilon_d/\varepsilon_{d+2}$ each time the distance grows by two. That same reference states $\Lambda \approx p_{\text{th}}/p$, which is precisely the identification implied by eq. (4.1) (a step $d \to d+2$ multiplies ε by $p/p_{\text{th}} = 1/\Lambda$); the two forms are therefore the same law.

The per-cycle rate is obtained from the measured per-experiment logical error rate p_L by dividing out the $r = d$ syndrome rounds (section 2.4), $\varepsilon = p_L/d$; the per-cycle logical error rate is the standard figure of merit in which Λ-suppression is quoted [11, 12]. Substituting into eq. (4.1) gives the fitted model `supp_model` exactly as implemented,

$$\ln\frac{p_L}{d} = \ln A + \kappa\, d\, \ln\frac{p}{p_{\text{th}}}, \tag{4.2}$$

in which $\ln A$ is the *only* free parameter (κ is fixed). The fit (`suppression_fit`) is a one-parameter `curve_fit` of $\ln(p_L/d)$ against the pair $(d,\ \ln(p/p_{\text{th}}))$, weighted by the propagated relative uncertainty $\sigma_{\ln} = \sigma/p_L$, with `absolute_sigma=True`. The threshold p_{th} is supplied by the FSS fit of chapter 3 (`estimate_threshold`).

4.2 The sub-threshold slice

The suppression law only holds below threshold, so the fit is restricted to a *sub-threshold slice* of each (η, code) group (`subthreshold_slice`): the points with at least one observed failure ($k > 0$) and physical rate $p < p_{\text{th}}$. A slice is used only if it retains at least 6 points spanning at least 3 distinct distances, so that the single-parameter line in eq. (4.2) is over-determined in d; otherwise the group is skipped. Requiring $k > 0$ keeps $\ln(p_L/d)$ finite, and the $p < p_{\text{th}}$ cut removes the above-threshold points where eq. (4.1) is invalid.

4.3 Required distance and physical-qubit count

Given the fitted $\ln A$, the target is a per-cycle logical error rate $\varepsilon_{\text{target}}$ at a fixed physical operating point p_{common} common to both codes. Inverting the suppression law to solve for the distance that meets a target error rate, and then counting the qubits it requires, is the standard surface-code resource extrapolation [9]; the same inversion underlies the Google Quantum AI projection that a suppression factor $\Lambda = 4$ and distance $d = 17$ (i.e. 577 physical qubits) reach a logical error rate of 10^{-6} [11]. Inverting eq. (4.1) for

the distance (`required_distance`),

$$d^* = \frac{\ln \varepsilon_{\text{target}} - \ln A}{\kappa \, \ln(p_{\text{common}}/p_{\text{th}})}, \tag{4.3}$$

and d^* is rounded *up* to the smallest odd integer ≥ 3. The denominator is negative only when $p_{\text{common}} < p_{\text{th}}$; if the operating point is not below threshold, or the fit does not suppress, the routine returns a sentinel -1 (no finite cost). A larger threshold makes $\ln(p_{\text{common}}/p_{\text{th}})$ more negative, shrinking d^* — this is the lever through which a higher XZZX threshold translates into a smaller code.

The physical-qubit cost is then the rotated-surface-code qubit count at that distance (`physical_qubits`) [18],

$$n = 2\,d^{*2} - 1 \qquad (d^{*2} \text{ data qubits plus } d^{*2} - 1 \text{ ancillas}), \tag{4.4}$$

counting one logical qubit's worth of physical qubits for a memory patch.

4.4 Uncertainty via parametric bootstrap

An error bar on n is obtained by a parametric bootstrap over the slice (`qubit_cost_row`). For each of n_{boot} replicas the per-point failure counts are resampled as $k_i^* \sim \text{Binomial}(N_i, \hat{p}_{L,i})$ (with a continuity floor of 0.5 failures to avoid $\ln 0$), the suppression line eq. (4.2) is refit, and d^* and n are recomputed. The reported n_{err} is the standard deviation of the resampled n^*, and $(n_{\text{lo}}, n_{\text{hi}})$ are its 16th and 84th percentiles — the non-parametric analogue of a $\pm 1\sigma$ interval, on the same single-σ convention as the Wilson bars (section 3.4). Because d^* is an integer rounded to the next odd value, the resampled n^* can be perfectly stable across replicas when the fit sits far from a rounding boundary; in that case the bootstrap interval legitimately collapses ($n_{\text{err}} = 0$, $n_{\text{lo}} = n_{\text{hi}} = n$), reflecting the discreteness of the distance rather than an absence of statistical error in $\ln A$.

4.5 Outputs

`qubit_cost_table` runs the above for every (η, code) group with the default operating point of table 4.1, writing a row per group to `qubit_cost.csv` (threshold, κ, d^*, n, its bootstrap bounds, and the slice point count). `render_qubit_cost` then draws `qubit_cost.pdf`, a two-panel summary against bias: the left panel plots n per code on a logarithmic axis, and the right panel plots the qubit-saving ratio $n_{\text{CSS}}/n_{\text{XZZX}}$, with a reference line at unity marking no advantage. The bias axis places $\eta \to \infty$ one decade past the largest finite bias, matching the threshold figure of section 5.3. The

Table 4.1: Default parameters of the qubit-cost estimate (`render_qubit_cost` / `qubit_cost_table`).

Parameter	Value
Suppression prefactor κ	0.5 (fixed)
Operating point p_{common}	10^{-3}
Target per-cycle rate $\varepsilon_{\text{target}}$	10^{-12}
Bootstrap replicas n_{boot}	500
Slice cut	$k > 0$ and $p < p_{\text{th}}$
Slice minimum	≥ 6 points, ≥ 3 distances
Qubit count	$n = 2{d^*}^2 - 1$

numerical results are reported in section 5.4.

Chapter 5

Results

This chapter reports the simulation results. The data are the representative sweep shipped under `full_results/` in the project repository, comprising 5856 Monte-Carlo points (all with $k > 0$), the fitted thresholds produced from them by the FSS procedure of chapter 3, and the physical-qubit cost derived from them by the procedure of chapter 4.

5.1 Sweep configuration

The sweep uses the circuit-level (biased SD6) model, X-memory, MWPM decoding, and $r = d$ rounds. Table 5.1 lists the grid as it appears in `full_results/samples.csv`. Code distances run $d \in \{5, 7, 9, 11, 13, 15, 17, 19\}$ for *every* (η, code) group. The physical-error-rate window is bias-adapted: each window is a narrow band of width 0.006 sampled at step $\Delta p = 10^{-4}$, centred on the expected crossing of that (η, code) group. The CSS windows and the low-bias XZZX windows sit near $p \in [0.003, 0.010]$, while the XZZX windows shift upward with bias, up to $[0.019, 0.025]$ at $\eta \to \infty$, tracking the rising XZZX threshold so that the crossing stays inside the sampled range. Each point is sampled adaptively up to a 300,000-shot cap; the error-count target is 30,000, and 2588 of the 5856 points reach the shot cap.

5.2 Logical error rate and data collapse figures

For each bias, the logical-versus-physical error-rate curves (`result_<eta>.pdf`) show the family of distances crossing near a common point, the visual threshold estimate of section 3.1, and the corresponding FSS data collapse (`collapse_<eta>.pdf`) rescales all fitted distances onto a single curve as evidence that the crossing is a genuine threshold. Figure 5.1 shows the pure-dephasing case $\eta \to \infty$, where the XZZX crossing sits at the highest physical error rate of the sweep.

Table 5.1: The representative sweep in `full_results/samples.csv`. Every window is 61 physical-rate values ($\Delta p = 10^{-4}$, width 0.006) across all eight distances, for 488 points per (η, code) group.

Bias η	Code	p window	n points
0.5	CSS	[0.004, 0.010]	488
1	CSS	[0.0035, 0.0095]	488
10, 30, 100, ∞	CSS	[0.003, 0.009]	488
0.5	XZZX	[0.004, 0.010]	488
1	XZZX	[0.0048, 0.0108]	488
10	XZZX	[0.010, 0.016]	488
30	XZZX	[0.013, 0.019]	488
100	XZZX	[0.016, 0.022]	488
∞	XZZX	[0.019, 0.025]	488

Table 5.2: Common sweep parameters (from the data and `config.py`).

Parameter	Value
Noise model	circuit-level (biased SD6)
Memory / decoder	X-memory / MWPM (PyMatching)
Distances d	$5, 7, 9, 11, 13, 15, 17, 19$
Biases η	$0.5, 1, 10, 30, 100, \infty$
Rounds	$r = d$
Shot cap	3×10^5
Error target	3×10^4
Error bar	Wilson score half-width, $z = 1$

5.3 Fitted thresholds versus bias

Table 5.3 lists the fitted thresholds for both codes across the six biases, together with the fitted critical exponent ν and the reduced chi-square of each fit, and fig. 5.3 plots the thresholds against η. The fit constrains ν to $[1, 2]$, the physically plausible band around the surface-code random-bond-Ising value $\nu \approx 1.5$ (section 3.2); across all (η, code) groups it settles near the lower end, $\nu \approx 1.09$–1.25, and the reduced chi-square stays close to unity, $\chi^2_{\text{red}} \approx 0.79$–$1.97$, with the largest values on the strongly biased XZZX fits where the bias-agnostic collapse degrades (section 3.8). The goodness-of-fit p-value of each fit is reported in its data-collapse figure (section 3.7); consistent with the near-unity χ^2_{red}, these p-values indicate statistically acceptable fits, and are smallest for the strongly biased XZZX cases that carry the largest χ^2_{red}.

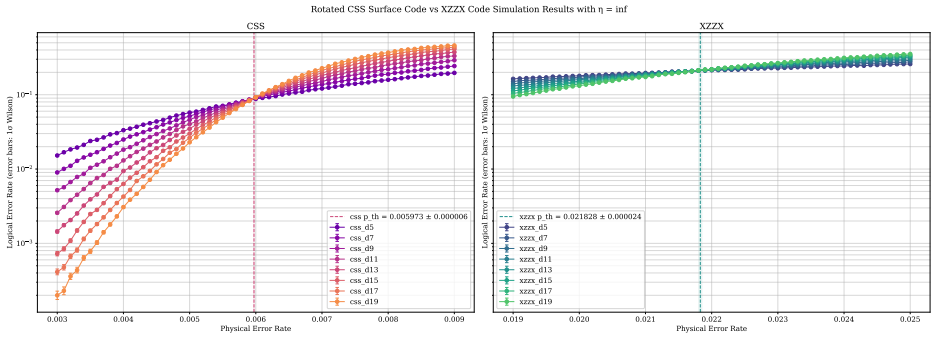

Figure 5.1: Logical error rate p_L versus physical error rate p at $\eta \to \infty$ (pure dephasing), for both codes across distances $d = 5$–19, with the fitted threshold line and 1σ band overlaid. The XZZX crossing lies well above the CSS crossing.

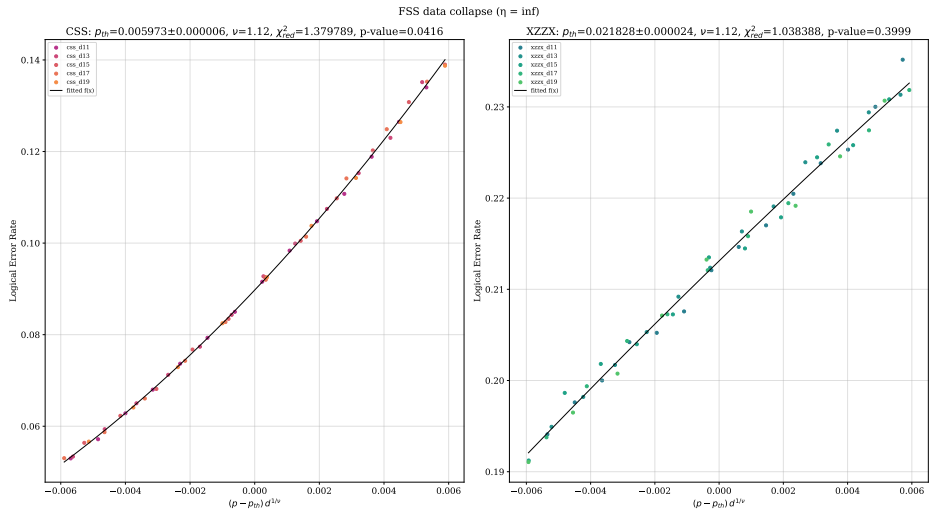

Figure 5.2: Finite-size-scaling data collapse at $\eta \to \infty$: each fitted-distance point is rescaled to $x = (p - p_{\text{th}})\, d^{1/\nu}$ and plotted against the logical error rate p_L inside the window $|x| < x_{\max}$, with the fitted parabola overlaid. A clean collapse of all distances is the evidence supporting the quoted p_{th} (section 3.7).

5.4 Physical-qubit cost

Feeding the thresholds and sub-threshold slices into the procedure of chapter 4 — with target per-cycle rate $\varepsilon_{\text{target}} = 10^{-12}$ at operating point $p_{\text{common}} = 10^{-3}$ — gives the physical-qubit cost of table 5.4, plotted against bias in fig. 5.4. The required distance d^* and the qubit count $n = 2{d^*}^2 - 1$ track the threshold: as the XZZX threshold climbs with bias, its operating point sits further below threshold, d^* shrinks, and the cost drops, while the near-flat CSS threshold leaves the CSS cost essentially bias-independent.

5.5 What the numbers show

Three features stand out and are the central result of the study:

Table 5.3: Fitted X-memory thresholds (circuit-level, MWPM) for the rotated CSS surface code and the XZZX code as a function of bias η, with the fitted critical exponent ν and the reduced chi-square χ^2_{red} of each fit. Thresholds are the project's reported FSS outputs (`full_results/figures/qubit_cost.csv`); ν and χ^2_{red} are read from the corresponding data-collapse fits (`collapse_<eta>.pdf`).

	CSS			XZZX		
Bias η	p_{th}	ν	χ^2_{red}	p_{th}	ν	χ^2_{red}
0.5 (depolarising)	0.687 %	1.09	1.04	0.721 %	1.09	1.23
1	0.660 %	1.12	1.12	0.786 %	1.12	1.26
10	0.606 %	1.12	1.04	1.230 %	1.11	1.54
30	0.599 %	1.13	0.79	1.510 %	1.17	1.18
100	0.599 %	1.11	1.14	1.792 %	1.25	1.97
∞ (pure Z)	0.597 %	1.12	1.38	2.183 %	1.12	1.04

Table 5.4: Physical-qubit cost per logical qubit to reach $\varepsilon_{\text{target}} = 10^{-12}$ per cycle at $p_{\text{common}} = 10^{-3}$, from `full_results/figures/qubit_cost.csv`. d^* is the required (odd) distance and $n = 2{d^*}^2 - 1$; the last column is the qubit-saving ratio $n_{\text{CSS}}/n_{\text{XZZX}}$.

	CSS		XZZX		
Bias η	d^*	n	d^*	n	$n_{\text{CSS}}/n_{\text{XZZX}}$
0.5 (depolarising)	25	1249	25	1249	1.00
1	25	1249	23	1057	1.18
10	27	1457	19	721	2.02
30	27	1457	19	721	2.02
100	27	1457	17	577	2.52
∞ (pure Z)	27	1457	17	577	2.52

- **Tied at low bias.** At $\eta = 0.5$ (depolarising) the two codes are essentially indistinguishable, ≈ 0.69–$0.72\,\%$, and cost the same 1249 physical qubits.
- **Growing XZZX advantage with bias.** As the noise becomes Z-biased, the XZZX threshold rises monotonically to $\approx 2.2\,\%$ at $\eta \to \infty$ — roughly a $3\times$ improvement over the depolarising point — while the CSS threshold stays flat and even drifts slightly down, to $\approx 0.60\,\%$. This is precisely the bias-tailoring advantage the XZZX Hadamard deformation is designed to deliver, and its consistency with the literature is examined in chapter 6.
- **Lower qubit cost.** The higher XZZX threshold translates directly into a smaller code: at strong bias the XZZX memory reaches the same target logical error rate with $\approx 2.5\times$ fewer physical qubits than the CSS surface code (577 versus 1457 at $\eta \to \infty$).

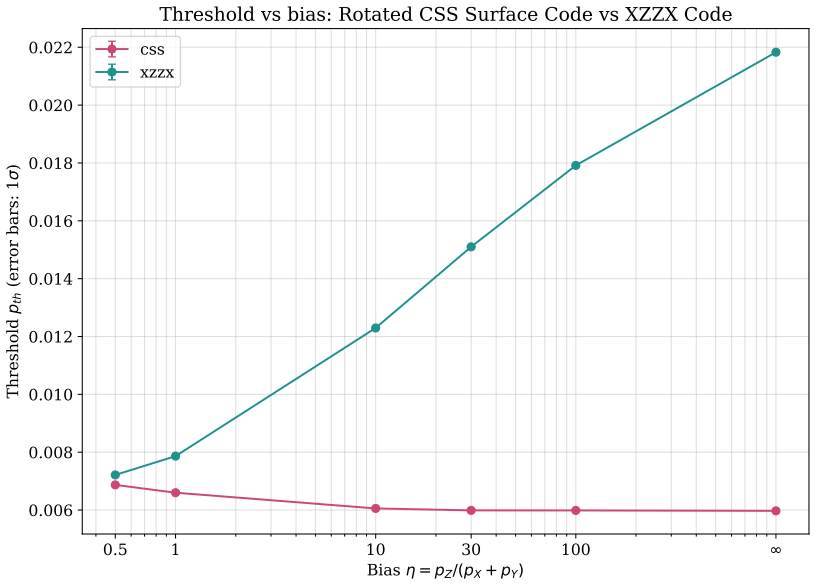

Figure 5.3: Fitted threshold $p_{\text{th}} \pm \delta$ versus bias η (log-scaled, with $\eta = \infty$ placed one decade past the largest finite bias) for both codes — the culminating CSS-versus-XZZX comparison. The XZZX threshold rises monotonically with bias while the CSS threshold stays essentially flat.

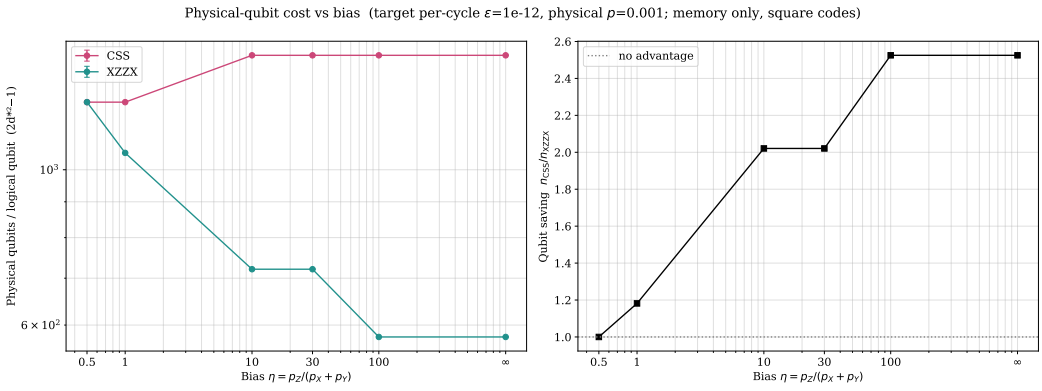

Figure 5.4: Physical-qubit cost versus bias (chapter 4). Left: physical qubits per logical qubit $n = 2d^{*2} - 1$ for each code (log axis). Right: qubit-saving ratio $n_{\text{CSS}}/n_{\text{XZZX}}$, with a reference line at unity. The XZZX code needs progressively fewer qubits as bias grows, reaching a $\approx 2.5\times$ saving at strong bias.

Chapter 6

Comparison with Prior Work, Consistency, and Limitations

This chapter situates the results of chapter 5 against the prior literature, records where they agree and where they should be read with care, and states the limitations of the study inherited from chapters 2 and 3.

6.1 Consistency with prior work

The XZZX advantage grows with bias; CSS does not. The monotonic separation of fig. 5.3 — an XZZX threshold that climbs with η while the CSS threshold stays flat — is the central qualitative result of Bonilla Ataides *et al.* [3] and, in the circuit-level / cat-qubit setting, of Darmawan *et al.* [4]. The origin, established already at code-capacity level by Tuckett *et al.* [19], is that the Hadamard deformation aligns the code so that pure-Z noise acts as effectively one-dimensional (repetition-code-like) errors. The present sweep reproduces this trend.

CSS depolarising threshold. The $\eta = 0.5$ CSS value ($\approx 0.69\,\%$) sits in the well-known band for the rotated surface code under circuit-level depolarising noise with MWPM (≈ 0.45–$0.5\,\%$ [9, 18], and the "Compare the Pair" study [20]). It runs slightly high for a specific, identifiable reason: this is a single-observable *X-memory* experiment, and at $\eta = 0.5$ the correlated two-qubit channel is *already mildly Z-biased*, because its depolarising point is $\eta = 1/4$ rather than $1/2$ (section 2.3.2). The X-memory therefore sees slightly less than its fair share of the noise, nudging the apparent threshold up.

XZZX infinite-bias threshold. The $\eta \to \infty$ XZZX value ($\approx 2.2\,\%$) is consistent with the $\approx 2.3\,\%$ circuit-level threshold reported for the XZZX code with bias-preserving CNOTs and MWPM in the biased-noise literature [4].

Not to be confused with code capacity. The frequently quoted "$\approx 50\,\%$ at infinite bias" figure for the XZZX code is a *code-capacity* number (the hashing bound of an effectively one-dimensional channel) [3]. The much smaller values here are *circuit-level* thresholds, where every operation is a fault location; the two live on different axes and should never be compared directly.

6.2 Contradictions and points of care

- **Absolute values are comparative, not precision determinations.** MWPM is sub-optimal for the correlated, biased noise (section 2.5), so each p_{th} is a lower bound, and the bias-agnostic FSS collapse (section 3.8) inflates χ^2_{red} for strongly biased XZZX. The reliable content is the *CSS-versus-XZZX* comparison within one model and one η, not the absolute thresholds.

- **Bias-preserving-gate assumption.** The uniform biased two-qubit channel presumes bias-preserving hardware (e.g. cat qubits [4]). On two-level qubits a no-go theorem forbids a bias-preserving CNOT, so a faithful transmon model would give the CX plain depolarising noise; the XZZX advantage would then *saturate* rather than keep growing with η. This is exactly the hybrid biased–depolarising regime of Etxezarreta Martinez *et al.* [8], where the residual CNOT bias saturates at $\eta_{\text{CNOT}} \approx 5$. The reported growth is therefore conditional on the hardware assumption and should be stated as such.

- **X-memory only.** The thresholds are X-memory values under Z-biased noise, not full (X and Z) code thresholds; the schedule is specialised to the single logical-X observable (section 2.4.4).

6.3 Limitations and future work

The limitations above point to a concrete programme for tightening the study:

- **Correlation-aware decoding.** Replacing MWPM with correlated matching or BP+OSD (section 2.5) would recover the residual correlations discarded by the graphlike DEM and raise the measured thresholds, most substantially for strongly biased XZZX; this is the single most direct improvement to the CSS-versus-XZZX comparison. The belief-matching [15] and Union-Find [5] backends are already wired in and can be exercised for this.

- **Bias-dependent scaling.** Letting the FSS scaling function or ν depend on η would remove the bias-agnostic-collapse limitation (section 3.8) and restore precision at strong bias.

- **Larger distances and denser sampling.** Extending beyond $d = 15$ and sampling more densely near each crossing would reduce the small-d finite-size systematic and bring χ^2_{red} closer to unity at higher shot budgets.
- **Two-sided and hardware-faithful models.** A full (X and Z) memory with the transposed schedule, and a transmon-faithful "hybrid" model with depolarising CX, would extend the comparison beyond the bias-preserving idealisation.

Appendix A

Complete Result Figures

This appendix collects the full set of per-bias result figures, of which chapter 5 shows only the $\eta \to \infty$ representative. Every figure is from the sweep of record in `full_results/` (circuit-level noise, X-memory, MWPM decoding, distances $d = 5$–19; section 5.1). For each bias η the upper panel is the logical-versus-physical error-rate curve `result_<eta>.pdf` (with the fitted threshold line and 1σ band), and the lower panel is the corresponding finite-size-scaling data collapse `collapse_<eta>.pdf` (section 3.7). The culminating threshold-versus-bias summary is fig. 5.3, and the physical-qubit cost summary fig. 5.4, both in chapter 5.

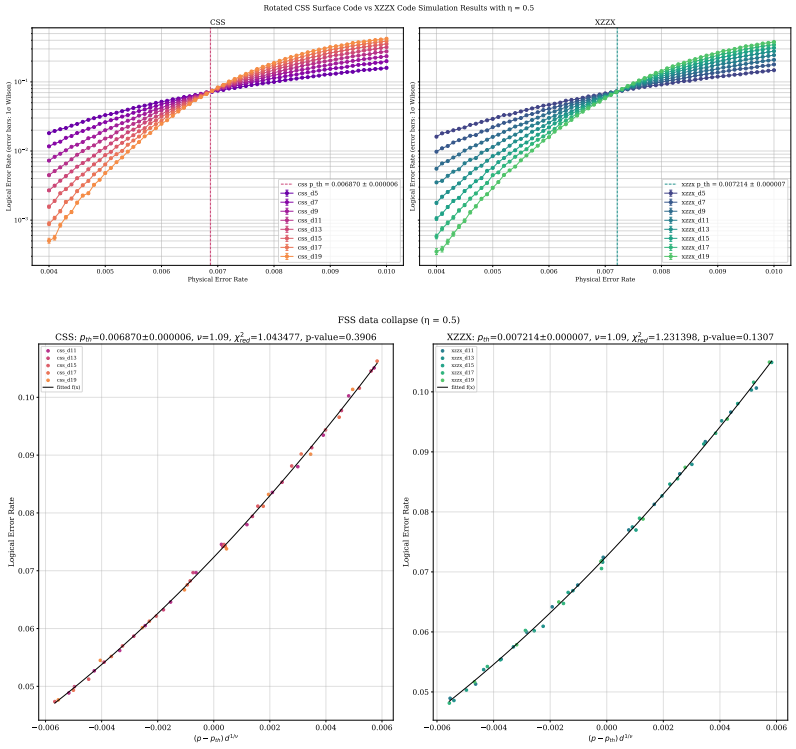

Figure A.1: Logical error rate (top) and FSS data collapse (bottom) at $\eta = 0.5$ (depolarising).

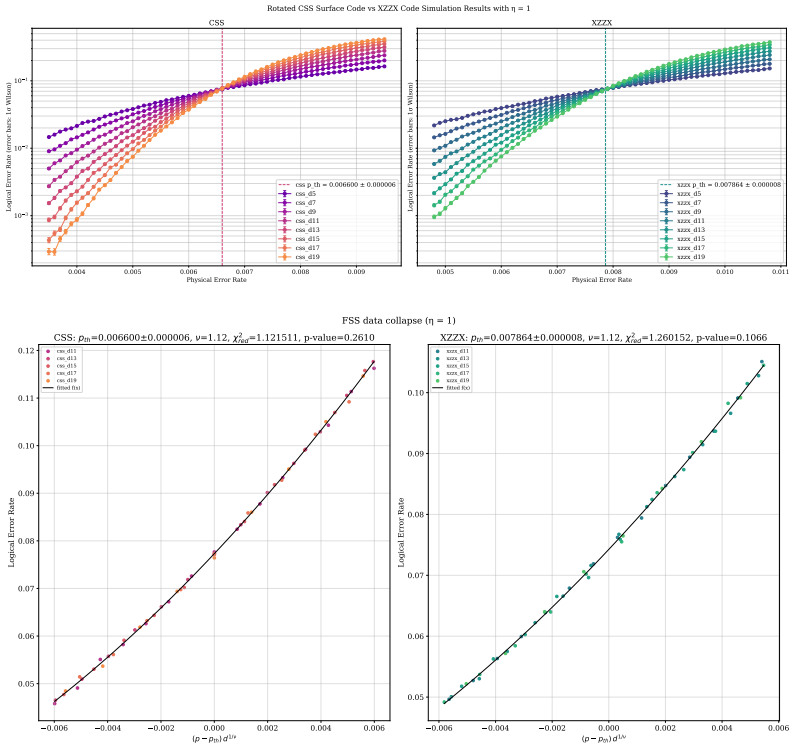

Figure A.2: Logical error rate (top) and FSS data collapse (bottom) at $\eta = 1$.

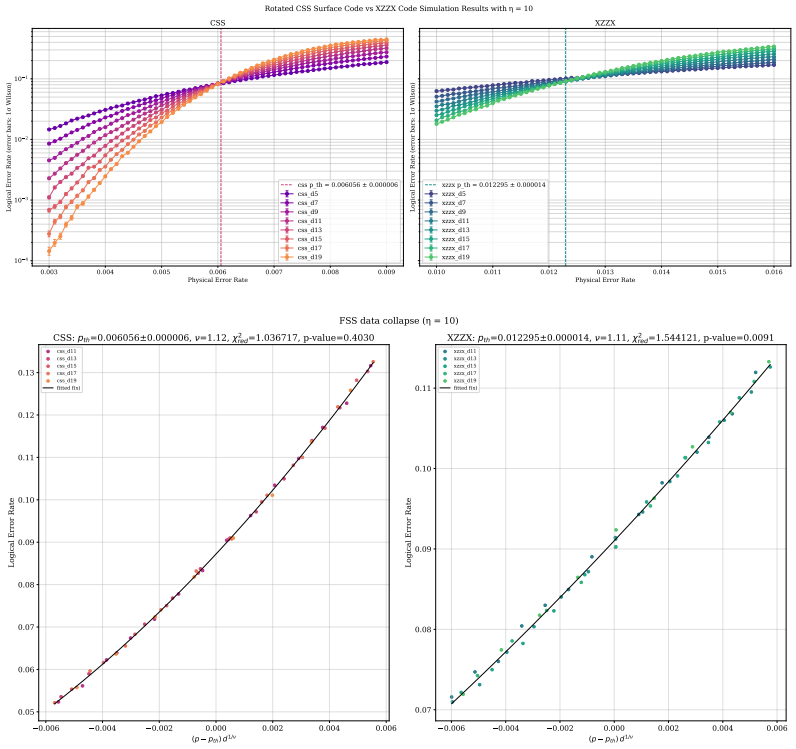

Figure A.3: Logical error rate (top) and FSS data collapse (bottom) at $\eta = 10$.

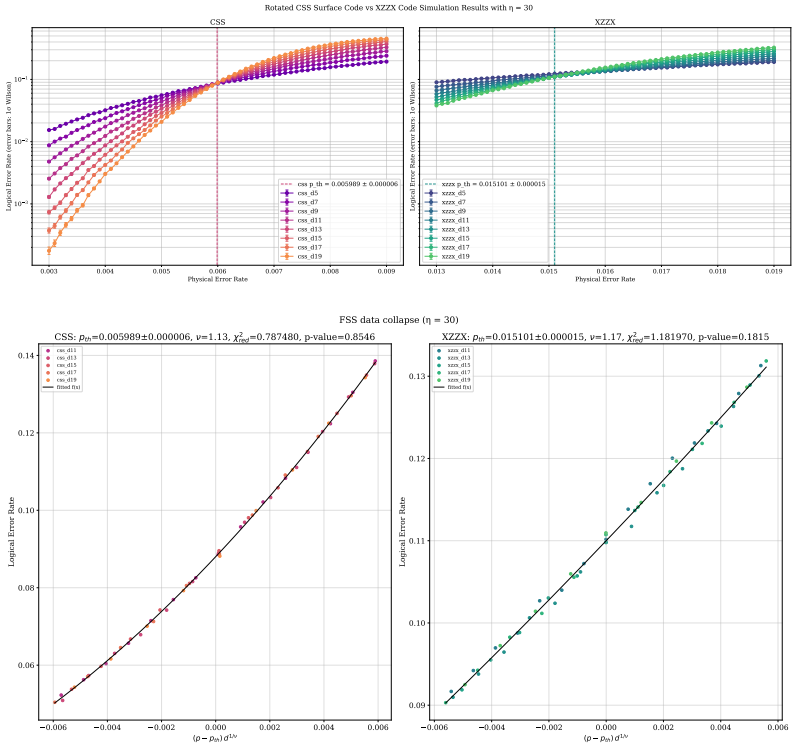

Figure A.4: Logical error rate (top) and FSS data collapse (bottom) at $\eta = 30$.

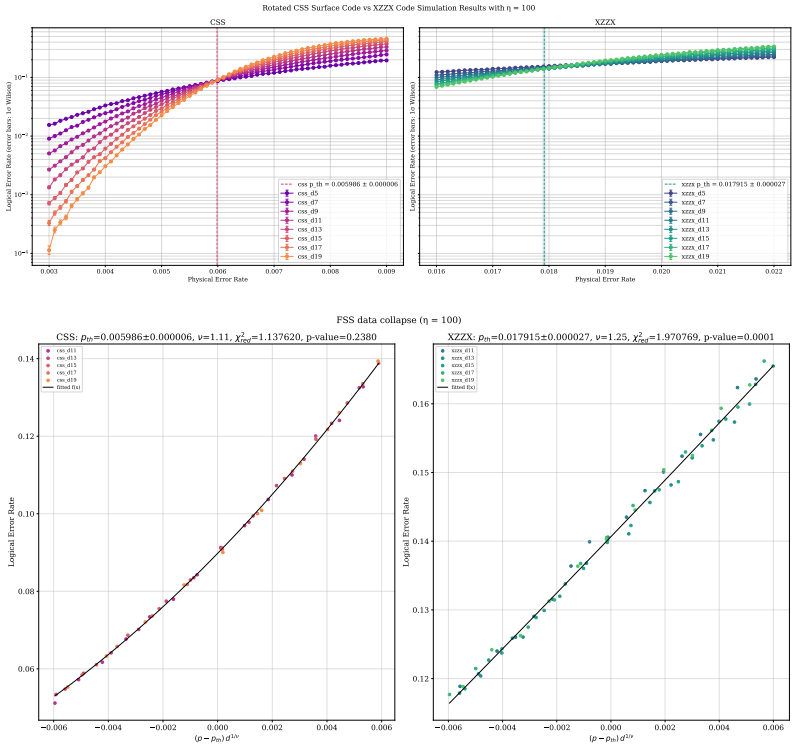

Figure A.5: Logical error rate (top) and FSS data collapse (bottom) at $\eta = 100$.

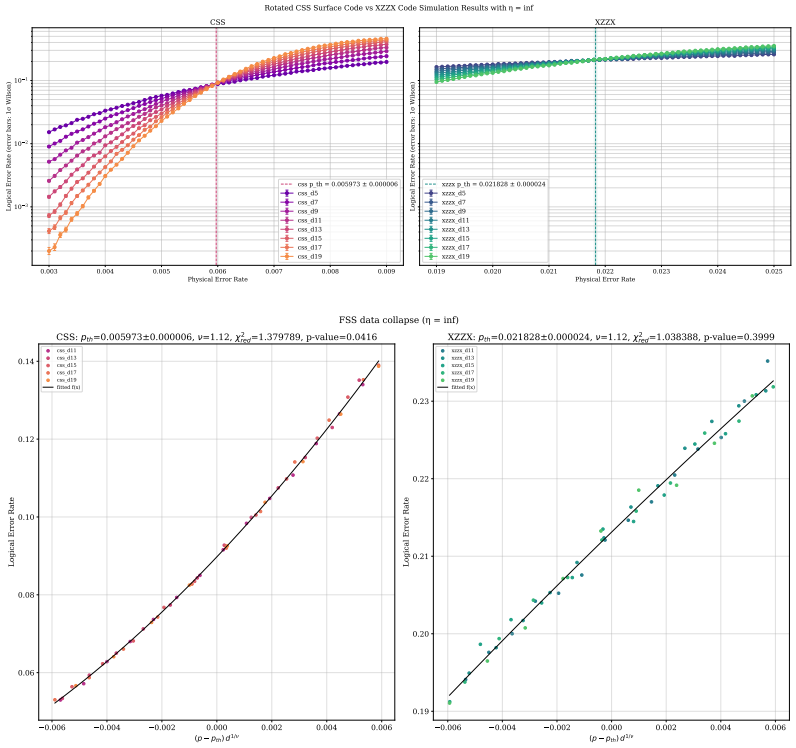

Figure A.6: Logical error rate (top) and FSS data collapse (bottom) at $\eta \to \infty$ (pure dephasing).

Bibliography

[1] Panos Aliferis and John Preskill. Fault-tolerant quantum computation against biased noise. *Phys. Rev. A*, 78(5):052331, 2008. doi: 10.1103/PhysRevA.78.052331. arXiv:0710.1301.

[2] H. Bombín, Ruben S. Andrist, Masayuki Ohzeki, Helmut G. Katzgraber, and M. A. Martin-Delgado. Strong resilience of topological codes to depolarization. *Phys. Rev. X*, 2(2):021004, 2012. doi: 10.1103/PhysRevX.2.021004.

[3] J. Pablo Bonilla Ataides, David K. Tuckett, Stephen D. Bartlett, Steven T. Flammia, and Benjamin J. Brown. The XZZX surface code. *Nature Communications*, 12:2172, 2021. doi: 10.1038/s41467-021-22274-1. arXiv:2009.07851.

[4] Andrew S. Darmawan, Benjamin J. Brown, Arne L. Grimsmo, David K. Tuckett, and Shruti Puri. Practical quantum error correction with the XZZX code and Kerr-cat qubits. *PRX Quantum*, 2(3):030345, 2021. doi: 10.1103/PRXQuantum.2.030345. arXiv:2104.09539.

[5] Nicolas Delfosse and Naomi H. Nickerson. Almost-linear time decoding algorithm for topological codes. *Quantum*, 5:595, 2021. doi: 10.22331/q-2021-12-02-595.

[6] Eric Dennis, Alexei Kitaev, Andrew Landau, and John Preskill. Topological quantum memory. *Journal of Mathematical Physics*, 43(9):4452–4505, 2002. doi: 10.1063/1.1499754.

[7] Jack Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17: 449–467, 1965. doi: 10.4153/CJM-1965-045-4.

[8] Josu Etxezarreta Martinez, Paul Schnabl, Javier Oliva del Moral, Reza Dastbasteh, Pedro M. Crespo, and Ruben M. Otxoa. Leveraging biased noise for more efficient quantum error correction at the circuit level with two-level qubits. *Phys. Rev. Applied*, 25(1):014021, 2026. doi: 10.1103/q7w6-nljp. arXiv:2505.17718.

[9] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland. Surface codes: Towards practical large-scale quantum computation. *Phys. Rev. A*, 86(3):032324, 2012. doi: 10.1103/PhysRevA.86.032324.

[10] Craig Gidney. Stim: a fast stabilizer circuit simulator. *Quantum*, 5:497, 2021. doi: 10.22331/q-2021-07-06-497.

[11] Google Quantum AI. Suppressing quantum errors by scaling a surface code logical qubit. *Nature*, 614(7949):676–681, 2023. doi: 10.1038/s41586-022-05434-1. arXiv:2207.06431.

[12] Google Quantum AI and Collaborators. Quantum error correction below the surface code threshold. *Nature*, 638(8052):920–926, 2025. doi: 10.1038/s41586-024-08449-y. arXiv:2408.13687.

[13] Oscar Higgott. PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching. *ACM Transactions on Quantum Computing*, 3(3):1–16, 2022. doi: 10.1145/3505637.

[14] Oscar Higgott and Craig Gidney. Sparse blossom: correcting a million errors per core second with minimum-weight matching. *Quantum*, 9:1600, 2025. doi: 10.22331/q-2025-01-20-1600. arXiv:2303.15933; VERIFY volume/page.

[15] Oscar Higgott, Thomas C. Bohdanowicz, Aleksander Kubica, Steven T. Flammia, and Earl T. Campbell. Improved decoding of circuit noise and fragile boundaries of tailored surface codes. *Phys. Rev. X*, 13(3):031007, 2023. doi: 10.1103/PhysRevX.13.031007. belief-matching decoder; arXiv:2203.04948.

[16] Shruti Puri, Lucas St-Jean, Jonathan A. Gross, Alexander Grimm, Nicholas E. Frattini, Pavithran S. Iyer, Anirudh Krishna, Steven Touzard, Liang Jiang, Alexandre Blais, Steven T. Flammia, and S. M. Girvin. Bias-preserving gates with stabilized cat qubits. *Science Advances*, 6(34):eaay5901, 2020. doi: 10.1126/sciadv.aay5901. arXiv:1905.00450.

[17] Joschka Roffe. LDPC: Python tools for low-density parity-check codes. `https://github.com/quantumgizmos/ldpc`, 2022. Union-Find decoder implementation; VERIFY citation preferred by the package.

[18] Yu Tomita and Krysta M. Svore. Low-distance surface codes under realistic quantum noise. *Phys. Rev. A*, 90(6):062320, 2014. doi: 10.1103/PhysRevA.90.062320.

[19] David K. Tuckett, Stephen D. Bartlett, and Steven T. Flammia. Ultrahigh error threshold for surface codes with biased noise. *Phys. Rev. Lett.*, 120(5):050505, 2018. doi: 10.1103/PhysRevLett.120.050505.

[20] [VERIFY authors] Compare the Pair collaboration. Compare the pair: rotated surface-code thresholds under circuit-level noise. *Phys. Rev. Research*, 7:033074, 2025. VERIFY full author list, exact title and DOI against the published article.

[21] Chenyang Wang, Jim Harrington, and John Preskill. Confinement-Higgs transition in a disordered gauge theory and the accuracy threshold for quantum memory. *Annals of Physics*, 303(1):31–58, 2003. doi: 10.1016/S0003-4916(02)00019-2.

[22] Yinzi Xiao, Basudha Srivastava, and Mats Granath. Exact results on finite size corrections for surface codes tailored to biased noise. *Quantum*, 8:1468, 2024. doi: 10.22331/q-2024-09-11-1468. arXiv:2401.04008.